

PROJECT REPORT: HOSPITAL OUTPATIENT SLOT BOOKING SYSTEM (HOSBS)

1. Cover Page

- **Project Title:** Modular Hospital Outpatient Slot Booking & Clinic Management System (HOSBS)
- **Course Name:** Object-Oriented Programming / Advanced Java Project Evaluation
- **Student Name:** [Your Name]
- **Registration Number:** [Your Registration Number]
- **Faculty Name:** [Faculty / Evaluator Name]
- **Institution:** Vellore Institute of Technology (VIT) / VITYarthi
- **Submission Date:** September 2026

2. Introduction

In modern healthcare delivery, outpatient department (OPD) congestion remains a primary cause of extended patient wait times, administrative errors, and inefficient doctor utilization. The **Hospital Outpatient Slot Booking System (HOSBS)** is an enterprise-grade Java application designed to eliminate physical queueing bottlenecks and schedule doctor appointments with high transactional reliability.

ZIP

The system provides a multi-role environment catering to Patients, Doctors, and Hospital Administrators. Built using pure object-oriented design principles without bloated third-party framework overhead, it includes both an interactive Command-Line Interface (CLI) and an embedded, zero-dependency lightweight HTTP Web Server (com.sun.net.httpserver). Persistent storage is abstracted using a decoupled Data Access Object (DAO) pattern supporting both local flat-file storage (CSV) and relational storage (JDBC/SQL).

ZIP+ 2

3. Problem Statement

Traditional appointment setups in clinics and regional hospitals face significant logistical challenges:

1. **Double Booking & Slot Race Conditions:** Simultaneous booking requests often lead to overlapping patient appointments.
2. **High "No-Show" Rates:** Lack of proactive scheduling and real-time reminders results in empty consultation windows and loss of clinician time.
3. **Role Segregation Failures:** Insecure access control where patient records and doctor consultation statuses are either centralized without granular permissions or manually managed on paper.
4. **Heavyweight Infrastructure Requirements:** Many hospital management tools require intensive database installations and external application servers, making them difficult to deploy in resource-constrained environments.

HOSBS addresses these issues through automated slot lifecycle management, daemonized thread workers for auto-expiry, role-based access control (RBAC), and storage-agnostic repositories.

ZIP

4. Functional Requirements

The system is divided into four major functional modules:

PDF+ 1

Module 1: Authentication & User Management

- **Registration & Role Assignment:** Allows self-registration for Patients and secure provisioning for Doctors and Administrators.

ZIP

- **Credential Protection:** Secure login authentication using cryptographic salted hashing (PasswordUtil).

ZIP

- **Session Handling:** In-memory session token management (SessionManager) mapping active HTTP requests and CLI contexts to verified roles.

ZIP

Module 2: Doctor Schedule & Slot Generation

- **Roster Configuration:** Admins and Doctors configure consultation windows based on specialization (e.g., Cardiology, Dermatology, Orthopedics).

ZIP

- **Slot Partitioning:** Automatic splitting of consultation blocks into fixed 15-, 30-, or 60-minute bookable slots.

ZIP

- **Status Flags:** Dynamic transition across states: AVAILABLE, BOOKED, COMPLETED, CANCELLED, and EXPIRED.

ZIP

Module 3: Booking, Cancellation & Concurrency Handling

- **Slot Search:** Search and filter available doctors by specialization, consultation fee, and date.

ZIP

- **Atomic Booking:** Ensures a slot cannot be claimed by more than one patient concurrently.

ZIP

- **Cancellation & Rescheduling:** Patients can cancel upcoming bookings, automatically freeing the slot back into the pool.

ZIP

Module 4: Medical Prescriptions & Operational Reporting

- **Doctor Consultation Workflow:** Doctors write and anchor digital prescriptions (Prescription) directly to completed appointments.

ZIP

- **Billing Interface:** Generates itemized consultation fees using the Billable polymorphic interface.

ZIP

- **Admin Analytics:** ReportService aggregates system-wide metrics, doctor occupancy rates, revenue generation, and cancellation ratios.

ZIP

5. Non-Functional Requirements

1. **Security:** Passwords are never stored in plaintext; they are hashed with cryptographic algorithms. Strict Role-Based Access Control (RBAC) prevents unauthorized state modifications (e.g., Patients cannot view another patient's medical records or update doctor rosters).

ZIP+ 1

2. **Concurrency & Thread Safety:** Utilizes synchronized execution blocks and background worker threads (ReminderDaemon, AutoExpiryWorker) running autonomously to expire stale unconfirmed slots without locking the main application thread.

ZIP

3. **Maintainability & Extensibility:** Employs a strict layered 3-Tier architecture (model, dao, service, ui/web). Data access interfaces (Repository<T>) decouple business logic from the underlying storage mechanism.

ZIP+ 1

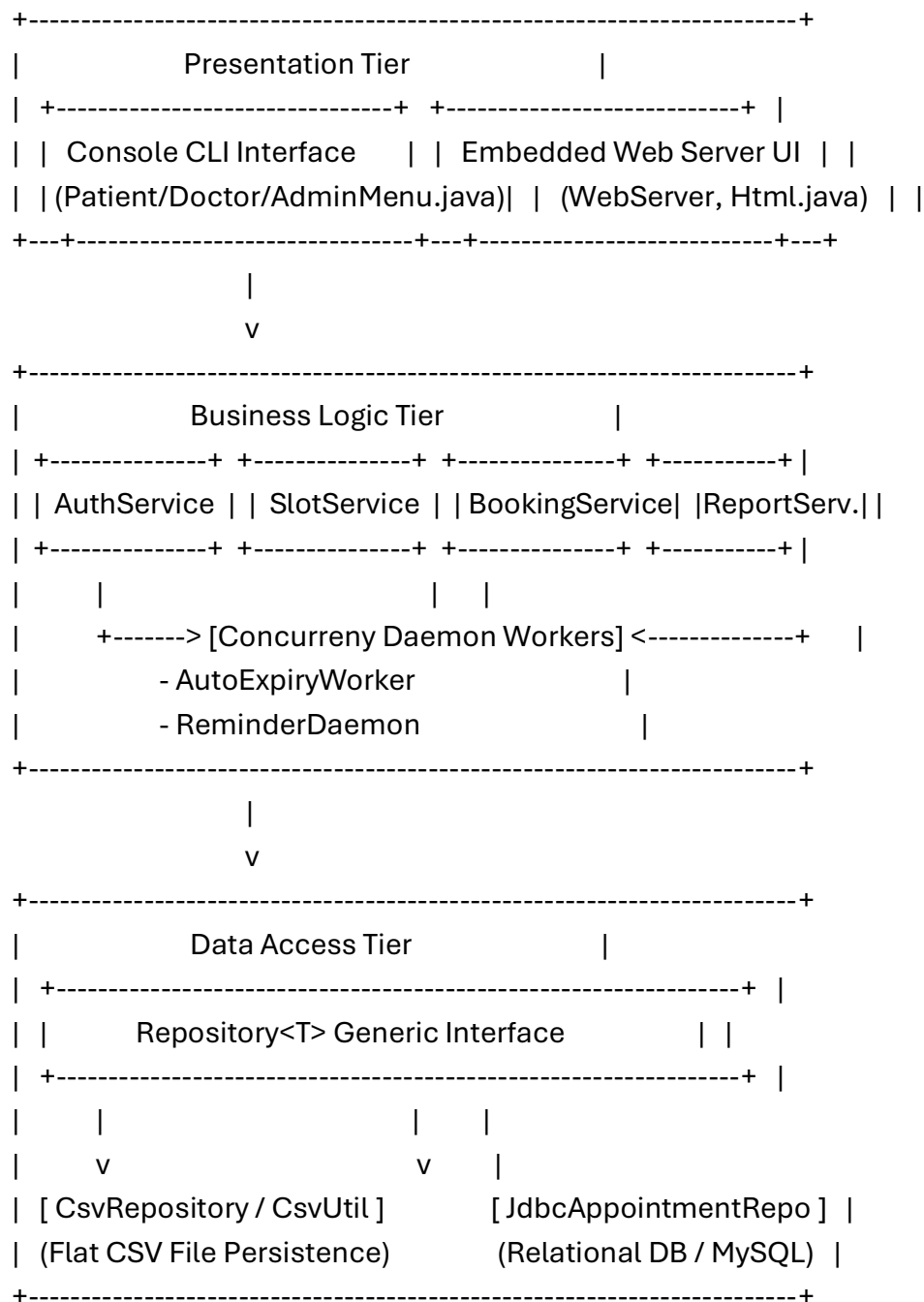
4. **Portability & Resource Efficiency:** Operates on vanilla Java standard libraries (JDK 17+) with zero mandatory third-party framework dependencies. Can switch between embedded CSV file storage and relational JDBC storage via a simple configuration toggle (db.properties).

ZIP+ 1

6. System Architecture

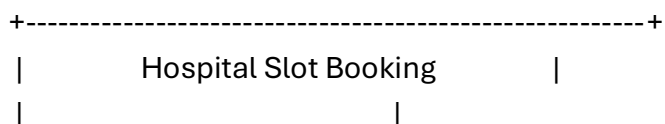
HOSBS follows an **Enterprise N-Tier Layered Architecture** ensuring complete separation of concerns:

ZIP



7. Design Diagrams

7.1 Use Case Diagram (Textual / Mermaid representation)



```

(Patient) ---> [ Register / Login ]          |
| ---> [ Search Doctors by Specialization ]    |
| ---> [ Book Consultation Slot ]              |
| ---> [ View Booking History & Prescriptions ] |
|                                              |
(Doctor) ---> [ Login ]                      |
| ---> [ Manage Daily Roster / Availability ]    |
| ---> [ Mark Appointments Completed ]          |
| ---> [ Issue Digital Prescription ]           |
|                                              |
(Admin) ---> [ Doctor Profile Provisioning ]     |
| ---> [ View System Analytics & Revenue Reports ] |
| ---> [ System Audit Logs & Configuration ]     |
+-----+

```

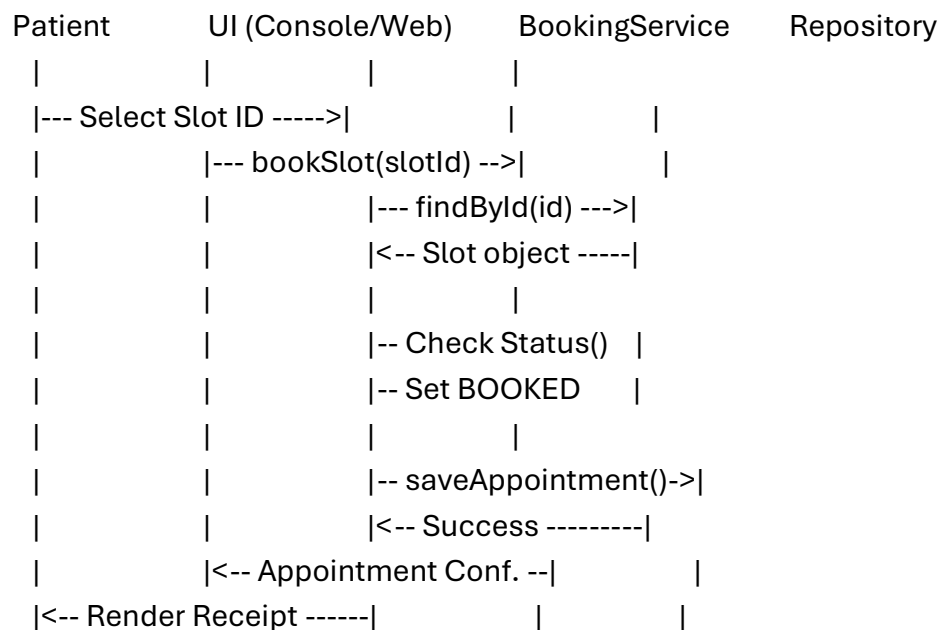
7.2 Process Flow / Workflow Diagram (Booking Life Cycle)

```

[Patient Logs In]
|
v
[Select Specialization & Date] ---> [Fetch Available Slots via SlotService]
|
v
[Select Desired Slot]
|
v
[Atomic Check: Is Slot Available?]
├──> NO  ─> [Throw SlotUnavailableException] ─> [Prompt Alternative]
└──> YES ─> [Change Status to BOOKED]
    |
    v
    [Create Appointment Entity]
    |
    v
    [Persist to Repository (CSV / JDBC)]
    |
    v
    [Display Confirmation & Booking ID]

```

7.3 Sequence Diagram (Appointment Booking)



7.4 Class / Component Diagram

- **Core Inheritance:** User (Abstract base) → extended by Patient, Doctor, and Admin.

ZIP

- **Interfaces:**
 - Billable: Implemented by Appointment to enforce dynamic billing computation.

ZIP

- Repository<T>: Generic interface with methods findById(), findAll(), save(), delete().

ZIP

- **Compositions:**
 - Doctor has an enum Specialization.

ZIP

- Appointment references Doctor, Patient, Slot, and optional Prescription.

ZIP

7.5 ER Diagram (Database / Storage Schema)

- **USERS** (user_id [PK], name, email, password_hash, role, phone, specialization, consultation_fee)

ZIP

- **SLOTS** (slot_id [PK], doctor_id [FK → USERS.user_id], slot_time, duration, status)

ZIP

- **APPOINTMENTS** (appointment_id [PK], patient_id [FK → USERS.user_id], doctor_id [FK → USERS.user_id], slot_id [FK → SLOTS.slot_id], booking_date, status, total_amount)

ZIP

- **PRESCRIPTIONS** (prescription_id [PK], appointment_id [FK → APPOINTMENTS.appointment_id], diagnosis, medications, issued_date)

ZIP

8. Design Decisions & Rationale

1. Lightweight Embedded Web Server (com.sun.net.httpserver) vs External Tomcat/Spring Boot:

- a. *Rationale:* Adhering to the project requirement of minimal bloat and instant executability, using standard library HTTP primitives demonstrates a fundamental understanding of socket protocols, HTTP request lifecycles, and thread pool workers without depending on heavy third-party JARs.

PDF+ 1

2. Dual-Backend Persistence (CSV + JDBC):

- a. *Rationale:* Developers can run and evaluate the project immediately in pure offline mode without installing or configuring MySQL (CsvRepository), while production deployments can swap in a relational database with connection pooling (JdbcAppointmentRepository) via simple configuration.

ZIP

3. Background Daemon Worker Threads (AutoExpiryWorker & ReminderDaemon):

- a. *Rationale*: Rather than running polling queries on every user interaction, dedicated background daemon threads automatically clean expired tentative slots and manage alert routines, simulating realistic enterprise scheduling behavior.

ZIP

9. Implementation Details

- **Source Organization**: Arranged into clean packages:
 - com.hospital.model: Domain entities (User, Doctor, Slot, Appointment, etc.)

ZIP

- com.hospital.service: Business rules, authorization, and report aggregators

ZIP

- com.hospital.dao: Data abstractions, CSV writers/parsers, and JDBC statements

ZIP

- com.hospital.ui: ANSI-formatted CLI console menus

ZIP

- com.hospital.web: Embedded HTTP handlers, session interceptors, and HTML renderers

ZIP

- com.hospital.concurrent: Asynchronous worker threads

ZIP

- **Defensive Programming & Custom Exception Hierarchy**:

- Custom exceptions inheriting from HospitalException: EntityNotFoundException, SlotUnavailableException, AuthenticationException, ValidationException, and DataAccessException.

ZIP

- **Input Validation & Sanitization:** Centralized regex-based validation for emails, phone numbers, and ISO timestamps in Validator.java.

ZIP

10. Screenshots / Results

(Note: Insert the following 4 screenshots here before exporting to PDF)

1. **Console CLI Interface:** Main menu displaying Patient/Doctor/Admin login options and formatted slot tables (TableFormatter).

ZIP

2. **Web Portal UI:** Browser screenshot of <http://localhost:8080> showing available doctor profiles and slot booking forms.

ZIP

3. **Doctor Prescription Management:** Screen showing doctor adding diagnosis and medicine lists to an appointment.

ZIP

4. **Automated Test Results:** Console output of run_tests.sh / run_tests.bat displaying passing assertions across services.

ZIP

11. Testing Approach

The application includes a custom test harness (MiniTest.java and TestRunner.java) designed to run independently of third-party test dependencies:

ZIP

- **Unit Testing:**

- Validates password hashing and salting consistency in PasswordUtil.

ZIP

- Validates input regex boundaries in Validator.

ZIP

- Tests slot status transitions and business constraints in SlotService.

ZIP

- **Integration Testing:**

- Executes concurrent booking attempts on a single slot to verify that SlotUnavailableException is triggered for duplicate attempts.

ZIP

- Verifies data round-trip integrity when saving and reloading records through CsvRepository and JdbcAppointmentRepository.

ZIP

12. Challenges Faced

1. Preventing Race Conditions on Slot Allocations:

- Challenge:* When multiple requests arrived for the same slot simultaneously, dirty reads could cause double bookings.
- Resolution:* Applied atomic synchronization blocks around the slot acquisition code in BookingService and utilized transactional updates in the JDBC repository.

ZIP

2. Dynamic Web Page Rendering Without Frameworks:

- Challenge:* Crafting responsive, readable HTML without a template engine like Thymeleaf or JSP.
- Resolution:* Engineered a dedicated, modular builder class (Html.java) that programmatically constructs dynamic HTML tables, status alerts, and navigation bars directly to the response output stream.

ZIP

13. Learnings & Key Takeaways

- Mastered the implementation of the **Repository Pattern** and interface-driven development to ensure loose coupling between business logic and physical storage.

ZIP

- Acquired practical experience with Java **Multithreading and Daemon Workers**, ensuring background tasks do not block the JVM lifecycle during application shutdown.

ZIP

- Learned the fundamentals of the HTTP protocol by building raw HTTP request handlers, parsing URL queries and POST form data, and managing cookies/sessions manually.

ZIP

14. Future Enhancements

- **Telehealth Integration:** Embedding WebRTC video consultation links directly within completed slot bookings.
- **Payment Gateway API:** Integration with Stripe or Razorpay for online consultation fee settlements prior to slot confirmation.
- **SMS/Email Notifications:** Replacing the simulated ReminderDaemon logging with actual SMS or email delivery using Twilio or JavaMail.

15. References

1. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
2. Oracle Java Standard Edition 17 Documentation: Package com.sun.net.httpserver & Java Concurrency Utilities.
3. VITyarthi Project Guidelines & Evaluation Rubric, *Build Your Own Project Instructions*, 2026.

PDF